

# Bazele Inteligenței Artificiale

## Lab 11. Arhitectura Transformer

Efrem Dragoș-Sebastian-Mihaly

# Ce vrem să facem azi

- Săptămâna trecută ne-am uitat peste RNN-uri și am văzut două mari dezavantaje:
  - ieșirea nu este flexibilă
  - memoria contextuală era scurtă
- Vrem să resolvăm aceste probleme și, de fapt, să ne uităm la inima care stă în spatele modelelor precum ChatGPT, Gemini, Claude & family
- Mai exact, astăzi vom explora arhitectura Transformer pentru a realiza un model care primește ca intrare text și generează ca ieșire text
- Există mai multe use cases pentru un astfel de model:
  - Question answering
  - Language translation
  - Chat bot, & altele

# Pregătirea datelor

- Să zicem că vrem să facem un model care știe să răspundă la întrebări
- Adică avem următoarele combinații de intrare - ieșire:

- Intrare

**If a red ball is larger than a blue ball, and the blue ball is larger than a green ball, which ball is the smallest?**

- Ieșire

**The question provides a hierarchy of sizes: the red ball is larger than the blue ball, and the blue ball is larger than the green ball. By comparing these relationships, we can determine the order of sizes as red > blue > green. Since the green ball is at the bottom of this hierarchy, it is the smallest among the three. Therefore, the answer is the green ball.**



# Pregătirea datelor

- Intrare

**If a red ball is larger than a blue ball, and the blue ball is larger than a green ball, which ball is the smallest?**

- Ieșire

The question provides a hierarchy of sizes: the red ball is larger than the blue ball, and the blue ball is larger than the green ball. By comparing these relationships, we can determine the order of sizes as  $\text{red} > \text{blue} > \text{green}$ . Since the green ball is at the bottom of this hierarchy, it is the smallest among the three. Therefore, the answer is the green ball.

- 
- Pentru antrenare, avem nevoie de multe astfel de combinații de intrare-ieșire, în funcție de ce problemă vrem să rezolvăm!

# Pregătirea datelor

- Intrare

**If a red ball is larger than a blue ball, and the blue ball is larger than a green ball, which ball is the smallest?**

- Ieșire

**The question provides a hierarchy of sizes: the red ball is larger than the blue ball, and the blue ball is larger than the green ball. By comparing these relationships, we can determine the order of sizes as red > blue > green. Since the green ball is at the bottom of this hierarchy, it is the smallest among the three. Therefore, the answer is the green ball.**

- 
- Spre exemplu, pentru language translation, am fi avut intrarea în limba sursă și ieșirea în limba destinație drept combinație intrare-ieșire

# Pregătirea datelor

- Intrare

**If a red ball is larger than a blue ball, and the blue ball is larger than a green ball, which ball is the smallest?**

- Ieșire

The question provides a hierarchy of sizes: the red ball is larger than the blue ball, and the blue ball is larger than the green ball. By comparing these relationships, we can determine the order of sizes as  $\text{red} > \text{blue} > \text{green}$ . Since the green ball is at the bottom of this hierarchy, it is the smallest among the three. Therefore, the answer is the green ball.

- 
- Transformerul este alcătuit din două componente: codificatorul pentru procesarea intrării, respectiv decodificatorul pentru obținerea ieșirii



# Codificatorul

- Intrare

**If a red ball is larger than a blue ball, and the blue ball is larger than a green ball, which ball is the smallest?**

- Pasul 1: Textul e preprocesat dacă este nevoie, să zicem că vrem să transformăm doar literele mari în litere mici
- Pasul 2: Textul e transformat în tokens

**[„if”, „a”, „red”, „ball”, „is”, „larger”, „than”, „a”, „blue”, „ball” „ , ”,  
„and”, „the”, „blue”, „ball”, „is”, „larger”, „than”, „a”, „green”, „ball” „ , ”  
„which”, „ball”, „is”, „the”, „smallest”, „?”]**

# Codificatorul

[„if”, „a”, „red”, „ball”, „is”, „larger”, „than”, „a”, „blue”, „ball” „ , ”,  
„and”, „the”, „blue”, „ball”, „is”, „larger”, „than”, „a”, „green”, „ball” „ , ”  
„which”, „ball”, „is”, „the”, „smallest”, „?”]

- Până în acest punct, trebuie să cunoaștem dimensiunea vocabularului nostru, adică cu ce cuvinte lucrăm, respectiv care sunt cuvintele unknown
- De asemenea, trebuie să știm pentru fiecare cuvânt și ID-ul lui
- Pasul 3: Având acestea spuse, transformăm fiecare cuvânt în ID-ul lui corespunzător

[2, 3, 4, 5, 6, 7, 8, 3, 9, 5, 10, 11, 12, 9, 5, 6, 7, 8, 3, 13, 5, 10, 14, 5, 6, 12, 15, 16]



# Codificatorul

[2, 3, 4, 5, 6, 7, 8, 3, 9, 5, 10, 11, 12, 9, 5, 6, 7, 8, 3, 13, 5, 10, 14, 5, 6, 12, 15, 16]

- Acum, vom realiza o mică rețea care va învăța pentru fiecare cuvânt din vocabular o reprezentare abstractă numită word embedding
- Mai bine spus, pentru fiecare cuvânt din vocabular se învață un vector care să reprezinte acel cuvânt
- Vom avea o matrice cu dimensiunea (lungime vocabular, dimensiunea word embedding-urilor)
- De exemplu, pentru un cuvântul cu ID-ul 2 din vocabular, și dimensiunea word embedding-urilor de 5, se poate învăța următoare reprezentare:

$$\mathbf{w}[2] = [1, 0.2, 0.5, 0.8, 0.3]$$

# Codificatorul

[2, 3, 4, 5, 6, 7, 8, 3, 9, 5, 10, 11, 12, 9, 5, 6, 7, 8, 3, 13, 5, 10, 14, 5, 6, 12, 15, 16]

- Ce trebuie să știm este că acești vectori sunt învățați la antrenare, prin backpropagation
- Inițial, ei sunt inițializați oarecum „random” dar nu vor exista 2 cuvinte cu același word embedding, deoarece cuvintele trebuie să fie unice
- Să zicem că dimensionalitatea word embedding-ului este aleasă ca 3, pentru simplitate
- Haideți să vedem cum se transformă textul de mai sus, după înlocuirea fiecărui cuvânt cu word embedding-ul corespunzător

# Codificatorul

```
[  
[0.2, 0.3, 0.5], # if  
[0.4, 0.6, 0.5], # a  
[0.5, 0.2, 0.3], # red  
[0.7, 0.8, 0.6], # ball  
[0.6, 0.1, 0.4], # is  
[0.9, 0.7, 0.5], # larger  
[0.3, 0.4, 0.2], # than  
[0.4, 0.6, 0.5], # a  
[0.8, 0.5, 0.7], # blue  
[0.7, 0.8, 0.6], # ball  
[0.2, 0.2, 0.2], # ,  
[0.4, 0.4, 0.4], # and  
[0.5, 0.5, 0.5], # the  
[0.8, 0.5, 0.7], # blue  
[0.7, 0.8, 0.6], # ball  
[0.6, 0.1, 0.4], # is  
[0.9, 0.7, 0.5], # larger  
[0.3, 0.4, 0.2], # than  
[0.4, 0.6, 0.5], # a  
[0.3, 0.6, 0.5], # green  
[0.7, 0.8, 0.6], # ball  
[0.2, 0.2, 0.2], # ,  
[0.7, 0.3, 0.6], # which  
[0.7, 0.8, 0.6], # ball  
[0.6, 0.1, 0.4], # is  
[0.5, 0.5, 0.5], # the  
[0.6, 0.2, 0.4], # smallest  
[0.1, 0.2, 0.3] # ?  
]
```

- Ajungem la reprezentările din stânga
- Haideți să ne uităm la reprezentarea obținută, dar mai ales la reprezentarea cuvintelor identice din aceeași propoziție, de exemplu la cuvântul „ball”
- În acest caz, ball primește mai multe sensuri, deși are același word embedding: uneori se referă la mingea roșie, altă dată la mingea albastră, uneori la mingea verde și în final, e folosit în contextul unei întrebări
- Dacă folosim același word embedding peste tot, modelul nu va face diferența între diferitele sensuri pe care le ia cuvântul pe parcursul propoziției



# Codificatorul

```
[  
[0.2, 0.3, 0.5], # if  
[0.4, 0.6, 0.5], # a  
[0.5, 0.2, 0.3], # red  
[0.7, 0.8, 0.6], # ball  
[0.6, 0.1, 0.4], # is  
[0.9, 0.7, 0.5], # larger  
[0.3, 0.4, 0.2], # than  
[0.4, 0.6, 0.5], # a  
[0.8, 0.5, 0.7], # blue  
[0.7, 0.8, 0.6], # ball  
[0.2, 0.2, 0.2], # ,  
[0.4, 0.4, 0.4], # and  
[0.5, 0.5, 0.5], # the  
[0.8, 0.5, 0.7], # blue  
[0.7, 0.8, 0.6], # ball  
[0.6, 0.1, 0.4], # is  
[0.9, 0.7, 0.5], # larger  
[0.3, 0.4, 0.2], # than  
[0.4, 0.6, 0.5], # a  
[0.3, 0.6, 0.5], # green  
[0.7, 0.8, 0.6], # ball  
[0.2, 0.2, 0.2], # ,  
[0.7, 0.3, 0.6], # which  
[0.7, 0.8, 0.6], # ball  
[0.6, 0.1, 0.4], # is  
[0.5, 0.5, 0.5], # the  
[0.6, 0.2, 0.4], # smallest  
[0.1, 0.2, 0.3] # ?  
]
```

- De fapt, ce vrem să facem este ca ordinea să conteze în propoziție
- Momentan, nu există un sens al ordinii în propoziție, dacă am avea ceva de genul „the cat chased the mouse”, ar fi tratat la fel ca „the mouse chased the cat”
- Adică ce vrem să facem este deși „ball” are intrinsec același word embedding pentru toate aparițiile lui, să facem cumva ca fiecare apariție a lui să introducă „noutate”
- Această noutate ne va ajuta să diferențiem între ball folosit în contextul red ball, blue ball, green ball sau dacă e folosit într-o întrebare



# Codificatorul

```
[  
[0.2, 0.3, 0.5], # if  
[0.4, 0.6, 0.5], # a  
[0.5, 0.2, 0.3], # red  
[0.7, 0.8, 0.6], # ball  
[0.6, 0.1, 0.4], # is  
[0.9, 0.7, 0.5], # larger  
[0.3, 0.4, 0.2], # than  
[0.4, 0.6, 0.5], # a  
[0.8, 0.5, 0.7], # blue  
[0.7, 0.8, 0.6], # ball  
[0.2, 0.2, 0.2], # ,  
[0.4, 0.4, 0.4], # and  
[0.5, 0.5, 0.5], # the  
[0.8, 0.5, 0.7], # blue  
[0.7, 0.8, 0.6], # ball  
[0.6, 0.1, 0.4], # is  
[0.9, 0.7, 0.5], # larger  
[0.3, 0.4, 0.2], # than  
[0.4, 0.6, 0.5], # a  
[0.3, 0.6, 0.5], # green  
[0.7, 0.8, 0.6], # ball  
[0.2, 0.2, 0.2], # ,  
[0.7, 0.3, 0.6], # which  
[0.7, 0.8, 0.6], # ball  
[0.6, 0.1, 0.4], # is  
[0.5, 0.5, 0.5], # the  
[0.6, 0.2, 0.4], # smallest  
[0.1, 0.2, 0.3] # ?  
]
```

- Această noutate pe care încercăm să o adăugăm se realizează prin positional encodings
- Ce au descoperit cercetătorii, e că se pot obține aceste valori unice prin intermediul funcțiilor sinus și cosinus

$$p_{i,2j} = \sin\left(\frac{i}{10000^{2j/d}}\right), \quad p_{i,2j+1} = \cos\left(\frac{i}{10000^{2j/d}}\right)$$

- În acest caz,  $i$  reprezintă indicele cuvântului în propoziție,  $j$  reprezintă indicele dimensiunii din word embedding și  $d$  reprezintă dimensionalitate word embedding-ului

# Codificatorul

[  
 [0.2, 0.3, 0.5], # if  
 [0.4, 0.6, 0.5], # a  
 [0.5, 0.2, 0.3], # red  
 [0.7, 0.8, 0.6], # ball  
 [0.6, 0.1, 0.4], # is  
 [0.9, 0.7, 0.5], # larger  
 [0.3, 0.4, 0.2], # than  
 [0.4, 0.6, 0.5], # a  
 [0.8, 0.5, 0.7], # blue  
 [0.7, 0.8, 0.6], # ball  
 [0.2, 0.2, 0.2], # ,  
 [0.4, 0.4, 0.4], # and  
 [0.5, 0.5, 0.5], # the  
 [0.8, 0.5, 0.7], # blue  
 [0.7, 0.8, 0.6], # ball  
 [0.6, 0.1, 0.4], # is  
 [0.9, 0.7, 0.5], # larger  
 [0.3, 0.4, 0.2], # than  
 [0.4, 0.6, 0.5], # a  
 [0.3, 0.6, 0.5], # green  
 [0.7, 0.8, 0.6], # ball  
 [0.2, 0.2, 0.2], # ,  
 [0.7, 0.3, 0.6], # which  
 [0.7, 0.8, 0.6], # ball  
 [0.6, 0.1, 0.4], # is  
 [0.5, 0.5, 0.5], # the  
 [0.6, 0.2, 0.4], # smallest  
 [0.1, 0.2, 0.3] # ?  
 ]

$$p_{i,2j} = \sin\left(\frac{i}{10000^{2j/d}}\right), \quad p_{i,2j+1} = \cos\left(\frac{i}{10000^{2j/d}}\right)$$

- De exemplu să luăm prima apariție a cuvântului „ball”: [0.7, 0.8, 0.6]
- Această apariție are indicele 3 în propoziția inițială
- Iterăm prin word embedding și actualizăm valorile:

$$0.7 \leftarrow 0.7 + \sin\left(\frac{3}{10000^{\frac{2 \cdot 0}{3}}}\right) = 0.7 + \sin(3/1) = 0.7 + 0.1 = 0.8$$

$$0.8 \leftarrow 0.8 + \cos\left(\frac{3}{10000^{\frac{2 \cdot 1}{3}}}\right) = 0.8 + \cos\left(\frac{3}{215.44}\right) = 0.8 + (-1.0) = -0.2$$

$$0.6 \leftarrow 0.6 + \sin\left(\frac{3}{10000^{\frac{2 \cdot 2}{3}}}\right) = 0.6 + \sin\left(\frac{3}{46415.89}\right) = 0.6 + 0.0 = 0.6$$

- Obținem embedding-ul actualizat [0.8, -0.2, 0.6]

# Codificatorul

[  
 [0.2, 0.3, 0.5], # if  
 [0.4, 0.6, 0.5], # a  
 [0.5, 0.2, 0.3], # red  
 [0.7, 0.8, 0.6], # ball  
 [0.6, 0.1, 0.4], # is  
 [0.9, 0.7, 0.5], # larger  
 [0.3, 0.4, 0.2], # than  
 [0.4, 0.6, 0.5], # a  
 [0.8, 0.5, 0.7], # blue  
 [0.7, 0.8, 0.6], # ball  
 [0.2, 0.2, 0.2], #,  
 [0.4, 0.4, 0.4], # and  
 [0.5, 0.5, 0.5], # the  
 [0.8, 0.5, 0.7], # blue  
 [0.7, 0.8, 0.6], # ball  
 [0.6, 0.1, 0.4], # is  
 [0.9, 0.7, 0.5], # larger  
 [0.3, 0.4, 0.2], # than  
 [0.4, 0.6, 0.5], # a  
 [0.3, 0.6, 0.5], # green  
 [0.7, 0.8, 0.6], # ball  
 [0.2, 0.2, 0.2], #,  
 [0.7, 0.3, 0.6], # which  
 [0.7, 0.8, 0.6], # ball  
 [0.6, 0.1, 0.4], # is  
 [0.5, 0.5, 0.5], # the  
 [0.6, 0.2, 0.4], # smallest  
 [0.1, 0.2, 0.3] #?  
 ]

$$p_{i,2j} = \sin\left(\frac{i}{10000^{2j/d}}\right), \quad p_{i,2j+1} = \cos\left(\frac{i}{10000^{2j/d}}\right)$$

- Haideți să luăm și a doua apariție a lui „ball”: [0.7, 0.8, 0.6]
- Această apariție are indicele 9 în propoziția inițială
- Iterăm prin word embedding și actualizăm valorile:

$$0.7 \leftarrow 0.7 + \sin\left(\frac{9}{10000^{\frac{2 \cdot 0}{3}}}\right) = 0.7 + \sin(9/1) = 0.7 + 0.4 = 1.1$$

$$0.8 \leftarrow 0.8 + \cos\left(\frac{9}{10000^{\frac{2 \cdot 1}{3}}}\right) = 0.8 + \cos\left(\frac{9}{215.44}\right) = 0.8 + (-0.9) = -0.1$$

$$0.6 \leftarrow 0.6 + \sin\left(\frac{9}{10000^{\frac{2 \cdot 2}{3}}}\right) = 0.6 + \sin\left(\frac{9}{46415.89}\right) = 0.6 + 0.0 = 0.6$$

- Obținem embedding-ul actualizat [1.1, -0.1, 0.6]



# Codificatorul

```
[  
[0.2, 0.3, 0.5], # if  
[0.4, 0.6, 0.5], # a  
[0.5, 0.2, 0.3], # red  
[0.7, 0.8, 0.6], # ball  
[0.6, 0.1, 0.4], # is  
[0.9, 0.7, 0.5], # larger  
[0.3, 0.4, 0.2], # than  
[0.4, 0.6, 0.5], # a  
[0.8, 0.5, 0.7], # blue  
[0.7, 0.8, 0.6], # ball  
[0.2, 0.2, 0.2], # ,  
[0.4, 0.4, 0.4], # and  
[0.5, 0.5, 0.5], # the  
[0.8, 0.5, 0.7], # blue  
[0.7, 0.8, 0.6], # ball  
[0.6, 0.1, 0.4], # is  
[0.9, 0.7, 0.5], # larger  
[0.3, 0.4, 0.2], # than  
[0.4, 0.6, 0.5], # a  
[0.3, 0.6, 0.5], # green  
[0.7, 0.8, 0.6], # ball  
[0.2, 0.2, 0.2], # ,  
[0.7, 0.3, 0.6], # which  
[0.7, 0.8, 0.6], # ball  
[0.6, 0.1, 0.4], # is  
[0.5, 0.5, 0.5], # the  
[0.6, 0.2, 0.4], # smallest  
[0.1, 0.2, 0.3] # ?  
]
```

- Deja putem să facem diferența dintre „ball” la indicele 3 și „ball” la indicele 9
- Am obținut [0.8, -0.2, 0.6] pentru prima apariție și [1.1, -0.1, 0.6] pentru a doua apariție
- Astfel, am reușit să adăugăm un sens al ordinii în propoziție: embedding-urile sunt diferite pentru că sunt folosite în diferite părți din propoziție
- Repetăm acest proces pentru toate cuvintele



# Codificatorul

[  
[0.2, 0.3, 0.5], # if  
[0.4, 0.6, 0.5], # a  
[0.5, 0.2, 0.3], # red  
[0.7, 0.8, 0.6], # ball  
[0.6, 0.1, 0.4], # is  
[0.9, 0.7, 0.5], # larger  
[0.3, 0.4, 0.2], # than  
[0.4, 0.6, 0.5], # a  
[0.8, 0.5, 0.7], # blue  
[0.7, 0.8, 0.6], # ball  
[0.2, 0.2, 0.2], # ,  
[0.4, 0.4, 0.4], # and  
[0.5, 0.5, 0.5], # the  
[0.8, 0.5, 0.7], # blue  
[0.7, 0.8, 0.6], # ball  
[0.6, 0.1, 0.4], # is  
[0.9, 0.7, 0.5], # larger  
[0.3, 0.4, 0.2], # than  
[0.4, 0.6, 0.5], # a  
[0.3, 0.6, 0.5], # green  
[0.7, 0.8, 0.6], # ball  
[0.2, 0.2, 0.2], # ,  
[0.7, 0.3, 0.6], # which  
[0.7, 0.8, 0.6], # ball  
[0.6, 0.1, 0.4], # is  
[0.5, 0.5, 0.5], # the  
[0.6, 0.2, 0.4], # smallest  
[0.1, 0.2, 0.3] # ?  
]



[  
[0.2, 1.3, 0.5], # if  
[1.2, 1.1, 0.5], # a  
[1.4, -0.2, 0.3], # red  
[0.8, -0.2, 0.6], # ball  
[-0.2, -0.6, 0.4], # is  
[-0.9, -1.0, 0.5], # larger  
[-1.4, -1.5, 0.2], # than  
[-1.8, -1.8, 0.5], # a  
[-2.0, -2.0, 0.7], # blue  
[-1.1, -0.8, 0.6], # ball  
[-2.0, -1.8, 0.2], # ,  
[-1.8, -1.5, 0.4], # and  
[-1.4, -1.0, 0.5], # the  
[-0.9, -0.6, 0.7], # blue  
[-0.2, -0.2, 0.6], # ball  
[0.5, 0.3, 0.4], # is  
[1.0, 0.8, 0.5], # larger  
[1.5, 1.4, 0.2], # than  
[1.8, 1.8, 0.5], # a  
[2.0, 2.0, 0.5], # green  
[2.1, 2.0, 0.6], # ball  
[2.0, 1.8, 0.2], # ,  
[1.8, 1.5, 0.6], # which  
[1.4, 1.0, 0.6], # ball  
[0.9, 0.6, 0.4], # is  
[0.2, 0.2, 0.5], # the  
[-0.5, -0.3, 0.4], # smallest  
[-1.0, -0.8, 0.3] # ?  
]

# Codificatorul

```
[
[0.2, 0.3, 0.5], # if
[0.4, 0.6, 0.5], # a
[0.5, 0.2, 0.3], # red
[0.7, 0.8, 0.6], # ball
[0.6, 0.1, 0.4], # is
[0.9, 0.7, 0.5], # larger
[0.3, 0.4, 0.2], # than
[0.4, 0.6, 0.5], # a
[0.8, 0.5, 0.7], # blue
[0.7, 0.8, 0.6], # ball
[0.2, 0.2, 0.2], # ,
[0.4, 0.4, 0.4], # and
[0.5, 0.5, 0.5], # the
[0.8, 0.5, 0.7], # blue
[0.7, 0.8, 0.6], # ball
[0.6, 0.1, 0.4], # is
[0.9, 0.7, 0.5], # larger
[0.3, 0.4, 0.2], # than
[0.4, 0.6, 0.5], # a
[0.3, 0.6, 0.5], # green
[0.7, 0.8, 0.6], # ball
[0.2, 0.2, 0.2], # ,
[0.7, 0.3, 0.6], # which
[0.7, 0.8, 0.6], # ball
[0.6, 0.1, 0.4], # is
[0.5, 0.5, 0.5], # the
[0.6, 0.2, 0.4], # smallest
[0.1, 0.2, 0.3] # ?
]
```



```
[
[0.2, 1.3, 0.5], # if
[1.2, 1.1, 0.5], # a
[1.4, -0.2, 0.3], # red
[0.8, -0.2, 0.6], # ball
[-0.2, -0.6, 0.4], # is
[-0.9, -1.0, 0.5], # larger
[-1.4, -1.5, 0.2], # than
[-1.8, -1.8, 0.5], # a
[-2.0, -2.0, 0.7], # blue
[-1.1, -0.8, 0.6], # ball
[-2.0, -1.8, 0.2], # ,
[-1.8, -1.5, 0.4], # and
[-1.4, -1.0, 0.5], # the
[-0.9, -0.6, 0.7], # blue
[-0.2, -0.2, 0.6], # ball
[0.5, 0.3, 0.4], # is
[1.0, 0.8, 0.5], # larger
[1.5, 1.4, 0.2], # than
[1.8, 1.8, 0.5], # a
[2.0, 2.0, 0.5], # green
[2.1, 2.0, 0.6], # ball
[2.0, 1.8, 0.2], # ,
[1.8, 1.5, 0.6], # which
[1.4, 1.0, 0.6], # ball
[0.9, 0.6, 0.4], # is
[0.2, 0.2, 0.5], # the
[-0.5, -0.3, 0.4], # smallest
[-1.0, -0.8, 0.3] # ?
]
```

- Cum facem un model care să fie de atent, de exemplu, că prima apariție a lui „larger” se referă la red ball si blue ball, nu și la a treia?
- Introducem mecanismul de atenție
- Vrem de fapt să facem modelul să „înțeleagă ce citește”

# Mecanismul de atenție

- Mecanismul de atenție vrea să asocieze un scor între diferitele cuvinte dintr-o propoziție
- Scopul este ca acea combinație de cuvinte care sunt relevante să aibă un scor mare, iar cele care sunt mai puțin relevante să aibă un scor mai mic
- În slide-ul precedent, vrem ca **prima apariție a lui „larger”** să aibă un scor mare cu **prima apariție a lui „ball”**, dar și **a doua apariție a lui „ball”**, pe când cu **a treia apariție a lui „ball”** vrem să aibă un scor cât se poate de mic



# Mecanismul de atenție

- Pentru început, fiecare word embedding văzut mai înainte este transformat în încă 3 word embedding-uri
- Această transformare se realizează, de obicei, prin straturi fully-connected / MLP
- Cele 3 noi reprezentări ale fiecărui cuvânt vor purta numele de: query, key & value
- Originea termenilor query, key & value vine de la modul în care se caută informații pe internet folosind search engines

# Mecanismul de atenție

- Originea termenilor query, key & value vine de la modul în care se caută informații pe internet folosind search engines
- Mai exact, query-ul înseamnă ce vrei tu să cauți, de exemplu „filme gratis online”
- Key reprezintă cuvintele cheie dintr-un site
- Value reprezintă rezultatele individuale date de un motor de căutare
- Query-ul tău este comparat cu mai multe cuvinte cheie (keys), și dacă acestea se potrivesc într-o anumită măsură, vei primi site-urile căutate (values) ca rezultat



# Mecanismul de atenție

- Mecanismul de atenție s-a inspirat de aici, dar ce vrea el să facă e să ia un word embedding după positional encoding și să-l transforme într-o combinație ponderată de celelalte cuvinte
- Scopul: ca un cuvânt să conțină informație contextuală, adică să știe către cine se referă acel cuvânt
- Să zicem că avem un cuvânt  $i$  din propoziție
- Primul lucru pe care îl facem e să calculăm scorul de atenție între cuvântul  $i$  și cuvântul  $j$  ca un produs scalar între query al cuvântului  $i$  și key cuvântului  $j$ :

$$a_{i,j} = q_i \cdot k_j$$

- Asta ne spune practic cât de mult cuvântul  $i$  se potrivește cu cuvântul  $j$  ca relevanță

# Mecanismul de atenție

- Să zicem că vrem că avem 1 propoziție cu 5 cuvinte și vrem să comparăm un cuvânt cu cele 5 (inclusiv cuvântul însuși)

$$a_{i,j} = q_i \cdot k_j$$

- Se obțin  $a_{i,1}, a_{i,2}, a_{i,3}, a_{i,4}, a_{i,5}$
- De obicei se împart aceste valori la  $\sqrt{d}$  unde  $d$  este dimensionalitatea embedding-urilor (oferă stabilitate în calculul gradientilor):  $\frac{a_{i,1}}{\sqrt{d}}, \frac{a_{i,2}}{\sqrt{d}}, \frac{a_{i,3}}{\sqrt{d}}, \frac{a_{i,4}}{\sqrt{d}}, \frac{a_{i,5}}{\sqrt{d}}$
- Se aplică softmax pentru a trece în spațiul probabilităților:

$$\{s_{i,1}, s_{i,2}, s_{i,3}, s_{i,4}, s_{i,5}\} = \text{softmax}\left(\left\{\frac{a_{i,1}}{\sqrt{d}}, \frac{a_{i,2}}{\sqrt{d}}, \frac{a_{i,3}}{\sqrt{d}}, \frac{a_{i,4}}{\sqrt{d}}, \frac{a_{i,5}}{\sqrt{d}}\right\}\right)$$

# Mecanismul de atenție

```
attention_weights = [  
  0.01, # 1: If (1%)  
  0.01, # 2: a (1%)  
  0.12, # 3: red (12%)  
  0.12, # 4: ball (12%)  
  0.05, # 5: is (5%)  
  0.10, # 6: larger (10%)  
  0.05, # 7: than (5%)  
  0.01, # 8: a (1%)  
  0.12, # 9: blue (12%)  
  0.12, # 10: ball (12%)  
  0.01, # 11: , (1%)  
  0.01, # 12: and (1%)  
  0.01, # 13: the (1%)  
  0.01, # 14: blue (1%)  
  0.01, # 15: ball (1%)  
  0.05, # 16: is (5%)  
  0.01, # 17: larger (1%)  
  0.05, # 18: than (5%)  
  0.01, # 19: a (1%)  
  0.01, # 20: green (1%)  
  0.01, # 21: ball (1%)  
  0.01, # 22: , (1%)  
  0.01, # 23: which (1%)  
  0.01, # 24: ball (1%)  
  0.05, # 25: is (5%)  
  0.01, # 26: the (1%)  
  0.01, # 27: smallest (1%)  
  0.01 # 28: ? (1%)  
]
```

- Dacă aplicăm pe textul nostru original, ca să comparăm prima apariție a lui „larger” cu celelalte cuvinte, am putea obține ceva de genul
- Practic ne spune că e relevant în proporție mare (12%) cu cuvintele red, ball și blue, ball (primele apariții) și, în rest, cu celelalte cuvinte aproximativ 1%
- Atenție! Modelul își va da seama de aceste scoruri după antrenare, adică când reușește să învețe o reprezentare adecvată pentru fiecare cuvânt (query, key)



# Mecanismul de atenție

- Până acum am vorbit de query și key, dar unde intervine vectorul value al fiecărui cuvânt?
- Intervine când vrem să obținem un nou vector asociat fiecărui cuvânt, care să conțină informație din toate celelalte cuvinte
- Mai exact, dacă de exemplu  $\{s_{i,1}, s_{i,2}, s_{i,3}, s_{i,4}, s_{i,5}\}$  sunt scorurile obținute, atunci noua reprezentare a cuvântului  $w_i$  devine:

$$w_i = w_i + s_{i,1} \cdot v_1 + s_{i,2} \cdot v_2 + s_{i,3} \cdot v_3 + s_{i,4} \cdot v_4 + s_{i,5} \cdot v_5$$

- Unde  $w_i$  e reprezentarea cuvântului după positional encoding
- Acum  $w_i$  va conține informație contextuală din toată propoziția, ponderată de scorul de atenție al fiecărui cuvânt

# Codificatorul

```
[  
  [0.5,-1.2,0.3], # if  
  [1.0,0.8,-0.5], # a  
  [1.5,-0.5,0.7], # red  
  [0.9,-0.3,0.8], # ball  
  [-0.1,-0.7,0.5], # is  
  [-1.0,-0.9,0.6], # larger  
  [-1.3,-1.4,0.3], # than  
  [-1.7,-1.7,0.6], # a  
  [-1.9,-1.9,0.8], # blue  
  [-1.0,-0.7,0.7], # ball  
  [-1.9,-1.7,0.3], # ,  
  [-1.7,-1.4,0.5], # and  
  [-1.3,-0.9,0.6], # the  
  [-0.8,-0.5,0.8], # blue  
  [-0.1,-0.1,0.7], # ball  
  [0.6,0.4,0.5], # is  
  [1.1,0.9,0.6], # larger  
  [1.6,1.3,0.3], # than  
  [1.9,1.9,0.6], # a  
  [2.1,2.1,0.6], # green  
  [2.2,2.1,0.7], # ball  
  [2.1,1.9,0.3], # ,  
  [1.9,1.6,0.7], # which  
  [1.5,1.1,0.7], # ball  
  [1.0,0.7,0.5], # is  
  [0.3,0.3,0.6], # the  
  [-0.6,-0.4,0.5], # smallest  
  [-1.1,-0.9,0.4] # ?  
]
```

- Practic, fiecare vector asociat unui cuvânt va fi înlocuit cu un alt vector, care combină informație din toate cuvintele din propoziție
- Astfel, fiecare cuvânt va avea acum un context în propoziție

# Mecanismul de atenție

$$w_i = w_i + s_{i,1} \cdot v_1 + s_{i,2} \cdot v_2 + s_{i,3} \cdot v_3 + s_{i,4} \cdot v_4 + s_{i,5} \cdot v_5$$

- De ce nu folosim direct reprezentările cuvintelor după positional encoding la ponderare, de ce există values?
- Haideți să vedem de fapt ce învață query, keys & values:
  - Query-ul spune: „Eu, cuvântul X, caut să aflu ceva despre... (de exemplu, cine acționează, unde se întâmplă ceva)
  - Keys zice: „Eu, cuvântul Y, pot oferi informații despre caracteristici precum cine, ce, unde, când.” Keys reprezintă modul prin care cuvântul Y se face “descoperit” de alt cuvânt care pune întrebări (query)
  - Values este conținutul efectiv pe care cuvântul Y îl oferă atunci când este “găsit” de un query potrivit. Dacă query și key se potrivesc, value este bucata de informație pe care o vei adăuga la reprezentarea inițială a cuvântului care întreabă, pentru a-l ajuta să înțeleagă contextul mai bine



# Codificatorul

```
[  
  [0.5,-1.2, 0.3], # if  
  [1.0, 0.8,-0.5], # a  
  [1.5,-0.5, 0.7], # red  
  [0.9,-0.3, 0.8], # ball  
  [-0.1,-0.7, 0.5], # is  
  [-1.0,-0.9, 0.6], # larger  
  [-1.3,-1.4, 0.3], # than  
  [-1.7,-1.7, 0.6], # a  
  [-1.9,-1.9, 0.8], # blue  
  [-1.0,-0.7, 0.7], # ball  
  [-1.9,-1.7, 0.3], # ,  
  [-1.7,-1.4, 0.5], # and  
  [-1.3,-0.9, 0.6], # the  
  [-0.8,-0.5, 0.8], # blue  
  [-0.1,-0.1, 0.7], # ball  
  [0.6, 0.4, 0.5], # is  
  [1.1, 0.9, 0.6], # larger  
  [1.6, 1.3, 0.3], # than  
  [1.9, 1.9, 0.6], # a  
  [2.1, 2.1, 0.6], # green  
  [2.2, 2.1, 0.7], # ball  
  [2.1, 1.9, 0.3], # ,  
  [1.9, 1.6, 0.7], # which  
  [1.5, 1.1, 0.7], # ball  
  [1.0, 0.7, 0.5], # is  
  [0.3, 0.3, 0.6], # the  
  [-0.6,-0.4, 0.5], # smallest  
  [-1.1,-0.9, 0.4] # ?  
]
```

- Acesta e mecanismul de self-attention
- Se spune „self” attention pentru că fiecare cuvânt din aceeași propoziție își învață contextul relativ la toate celelalte cuvinte din propoziția respectivă
- După ce se obține această variantă a vectorilor, ce se întâmplă de obicei este să se transforme toți vectorii astfel încât media valorilor din fiecare vector să fie 0 și deviația standard să fie 1
- Lucrul acesta e făcut pentru a stabiliza procesul de antrenare, mai exact să nu se ajungă la gradienti prea mari sau prea mici

# Codificatorul

```
[
  [0.83, -1.41, 0.57], # if
  [0.85, 0.55, -1.40], # a
  [1.07, -1.34, 0.27], # red
  [0.80, -1.41, 0.61], # ball
  [0.00, -1.22, 1.22], # is
  [-0.77, -0.64, 1.41], # larger
  [-0.64, -0.77, 1.41], # than
  [-0.71, -0.71, 1.41], # a
  [-0.71, -0.71, 1.41], # blue
  [-0.90, -0.49, 1.39], # ball
  [-0.81, -0.60, 1.41], # ,
  [-0.86, -0.55, 1.40], # and
  [-0.94, -0.45, 1.39], # the
  [-0.91, -0.48, 1.39], # blue
  [-0.71, -0.71, 1.41], # ball
  [1.22, -1.22, 0.00], # is
  [1.14, 0.16, -1.30], # larger
  [0.96, 0.42, -1.38], # than
  [0.71, 0.71, -1.41], # a
  [0.71, 0.71, -1.41], # green
  [0.78, 0.63, -1.41], # ball
  [0.83, 0.58, -1.41], # ,
  [0.98, 0.39, -1.37], # which
  [1.22, 0.00, -1.22], # ball
  [1.30, -0.16, -1.14], # is
  [-0.71, -0.71, 1.41], # the
  [-0.91, -0.49, 1.39], # smallest
  [-0.85, -0.55, 1.40] # ?
]
```

```
[
  [-0.33, 0.90, -0.45], # if
  [-3.24, 0.16, -0.80], # a
  [-1.31, 1.79, 0.08], # red
  [1.12, 1.24, 1.35], # ball
  [-1.64, 1.66, 1.07], # is
  [-0.63, -0.51, 1.88], # larger
  [2.42, 0.46, -0.59], # than
  [-0.66, 0.04, 0.80], # a
  [0.79, -0.82, 1.56], # blue
  [2.12, 1.33, 1.75], # ball
  [-0.09, 0.75, 0.28], # ,
  [-0.95, 0.76, -0.47], # and
  [1.18, 1.19, -1.01], # the
  [2.11, 1.58, 0.34], # blue
  [0.48, -1.92, 0.23], # ball
  [0.16, -0.61, 0.19], # is
  [-0.02, 0.20, -0.30], # larger
  [0.27, -1.12, 0.08], # than
  [-1.76, -0.79, 0.13], # a
  [-0.35, -0.08, 1.04], # green
  [-0.83, -1.01, 1.66], # ball
  [-0.56, -0.35, 0.43], # ,
  [-0.24, 0.32, -0.88], # which
  [-0.96, -0.14, -1.25], # ball
  [-0.89, -0.05, 1.24], # is
  [-0.31, 0.61, 2.09], # the
  [-0.64, 0.83, 0.55], # smallest
  [-1.41, -0.60, 0.63] # ?
]
```

- Gata! Avem reprezentarea finală a fiecărui cuvânt de intrare
- Totuși, de obicei, atunci când întâlnim propoziții complexe, este considerat că o singură reprezentare pentru fiecare cuvânt s-ar putea să nu fie suficientă
- Astfel este introdus conceptul de multi-head self-attention: nu se obține o singură reprezentare pentru fiecare cuvânt, se obține un număr de N reprezentări
- N mai poartă numele și de „heads” sau „capete”
- în stânga, numărul de heads este 2

# Decodificatorul

- Decodificatorul poate fi văzut din două puncte de vedere:
  - al antrenării
  - al inferenței (realizării de preziceri)
- Scopul lui este să ne spună care este următorul cuvânt de generat
- Haideți să vedem la antrenare, primii pași sunt exact ca la codificator:





# Decodificatorul

**Positional Encoded  
Word Embeddings**

**Multi-Head Self-  
Attention\***

```
[  
  [0.8, 0.4, -0.6], # The  
  [0.7, -0.3, -0.4], # question  
  [1.1, 0.1, -1.2], # provides  
  [1.3, -0.5, -0.8], # a  
  [0.9, 0.8, -1.7], # hierarchy  
  [0.4, -1.1, 0.7], # of  
  [1.5, -0.4, 0.5], # sizes:  
  [-0.6, -0.8, 1.4], # the  
  [-0.2, -1.0, 1.2], # red  
  [0.1, 0.9, -1.0], # ball  
  [0.3, 1.5, -1.8], # is  
  [1.6, -0.2, -1.4], # larger  
  [-0.9, 1.3, 0.4], # than  
  [.....]  
  [1.1, -0.5, -1.0], # smallest  
  [1.3, -1.2, 0.1], # the  
  [0.8, -0.5, -1.1], # answer  
  [-0.7, 0.6, -1.3], # is  
  [-1.1, 1.0, 0.1], # the  
  [-1.4, -0.6, -0.8], # green  
  [-1.2, 0.4, 1.3] # ball.  
]
```

- Cu \* la self-attention pentru că, de exemplu, se întâmplă asta:

$$w_3 = w_3 + s_{3,1} \cdot v_1 + s_{3,2} \cdot v_2 + s_{3,3} \cdot v_3$$

- Mai exact, un cuvânt va fi codificat doar pe baza cuvântului în sine și pe cele care sunt înaintea lui, dar nu și prin cele care vin după el
- Asta pentru că vrem să învățăm modelul să genereze text în manieră secvențială, și în acest caz ai acces doar la textul până în momentul curent, nu și la cel din viitor



# Decodificatorul

**Positional Encoded  
Word Embeddings**



**Multi-Head Self-  
Attention\***

- Trebuie să avem un mod prin care să introducem informația de la codificator la decodificator
- Pentru că prin intermediul intrării vrem să generăm ieșirea, spre exemplu să generăm răspunsul la o întrebare
- Astfel, se introduce mecanismul de cross-attention
- Este foarte simplu: dacă la self-attention fiecare cuvânt era reprezentat ca o pondere din celelalte cuvinte din aceeași propoziție pentru a adăuga context, la cross-attention fiecare cuvânt din decodificator (de la ieșire) va fi reprezentat și ca o pondere a intrării

# Decodificatorul



- Practic, acum query-urile folosite sunt cele de la fiecare cuvânt de ieșire, iar keys & values sunt date de cuvintele de intrare

**Intrarea**

**Ieșirea (până la albastru)**

[  
 [0.83, -1.41, 0.57], # if  
 [0.85, 0.55, -1.40], # a  
 [1.07, -1.34, 0.27], # red  
 [0.80, -1.41, 0.61], # ball  
 [0.00, -1.22, 1.22], # is  
 [-0.77, -0.64, 1.41], # larger  
 [-0.64, -0.77, 1.41], # than  
 [-0.71, -0.71, 1.41], # a  
 [-0.71, -0.71, 1.41], # blue  
 [-0.90, -0.49, 1.39], # ball  
 [-0.81, -0.60, 1.41], #,  
 [-0.86, -0.55, 1.40], # and  
 [-0.94, -0.45, 1.39], # the  
 [-0.91, -0.48, 1.39], # blue  
 [-0.71, -0.71, 1.41], # ball  
 ]

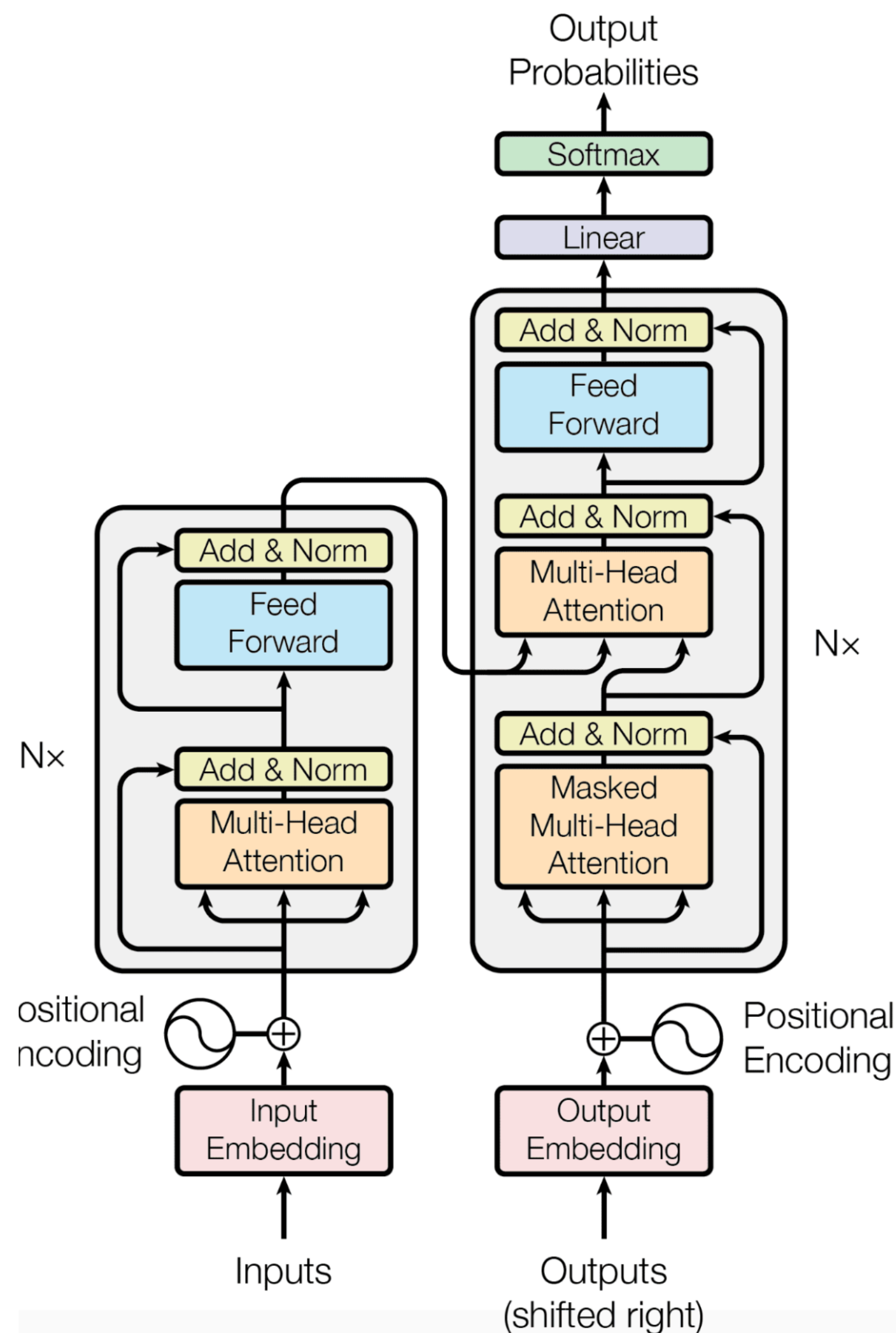
[  
 [0.8, 0.4, -0.6], # The  
 [0.7, -0.5, -0.4], # question  
 [1.1, 0.1, -1.2], # provides  
 [1.3, -0.5, -0.8], # a  
 [0.9, 0.8, -1.7], # hierarchy  
 [0.4, -1.1, 0.7], # of  
 [1.5, -0.4, 0.5], # sizes:  
 [-0.6, -0.8, 1.4], # the  
 [-0.2, -1.0, 1.2], # red  
 [0.1, 0.9, -1.0], # ball  
 [0.3, 1.5, -1.8], # is  
 [1.6, -0.2, -1.4], # larger  
 [-0.9, 1.3, 0.4], # than  
 [.....]  
 [1.1, 0.5, -1.0], # smallest  
 ]

$$w_3 = w_3 + (s_{3,1}^{self} \cdot v_1^{self} + s_{3,2}^{self} \cdot v_2^{self} + s_{3,3}^{self} \cdot v_3^{self}) + s_{3,1}^{cross} \cdot v_1^{cross} + s_{3,2}^{cross} \cdot v_2^{cross} + \dots + s_{3,n}^{cross} \cdot v_n^{cross}$$

- La ce am obținut prin **self-attention**, se adaugă termenii obținuți prin **cross-attention**
- $v_1^{cross}, v_2^{cross}, \dots, v_n^{cross}$  reprezintă values ale cuvintelor de intrare (unde lungime intrării este N)

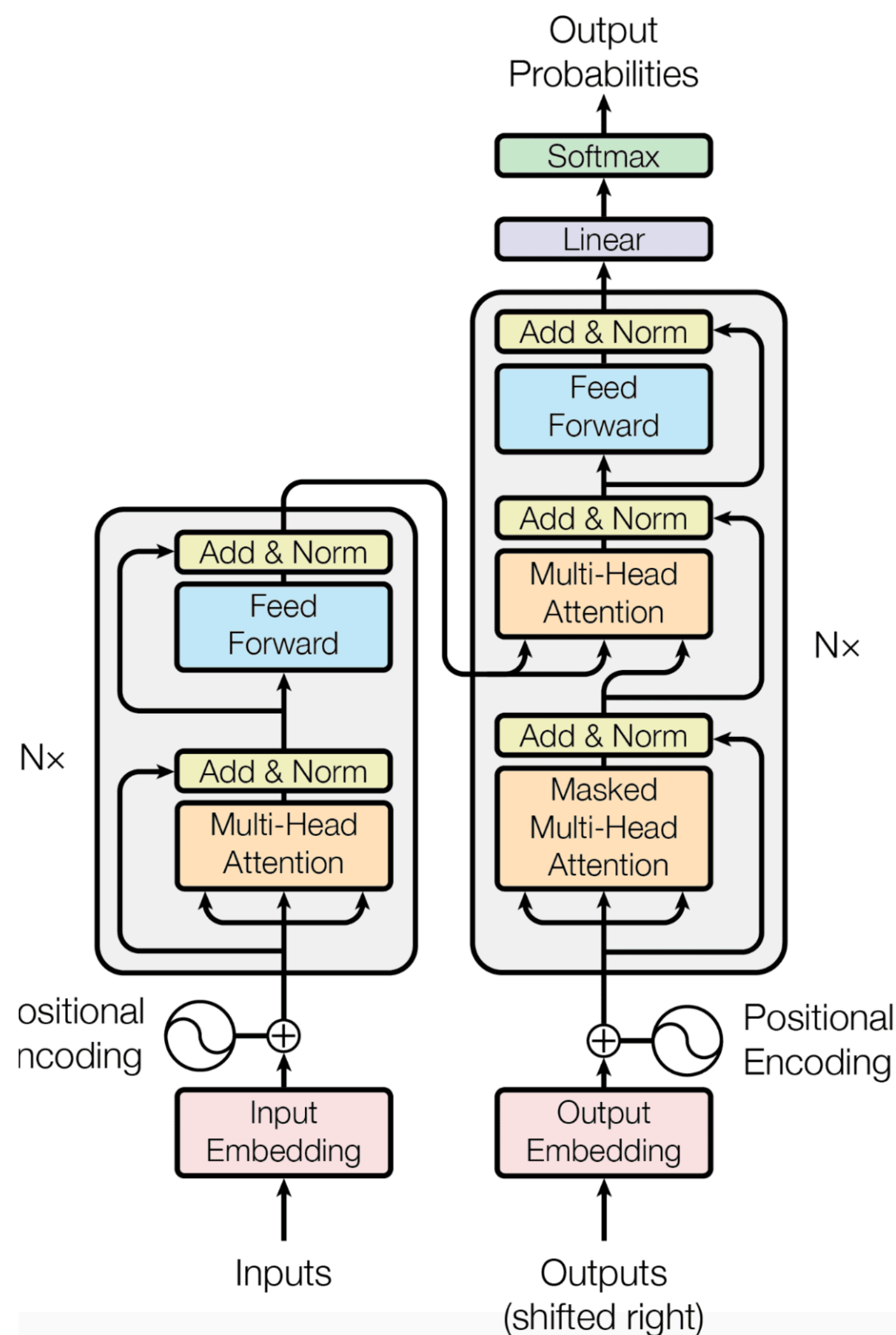


# Arhitectura Transformer



- De obicei, se folosește un Feed-Forward Network (MLP) după operațiile de atenție ca să se învețe reprezentări și mai complexe ale vectorilor de până acum
- La decodificator, vectorul ultimului cuvânt generat până acum este transmis către un strat liniar (fully-connected), care are rolul de a transforma reprezentarea abstractă a cuvântului în spațiul vocabularului
- Peste acel spațiu este aplicat softmax, pentru a trece în spațiul probabilităților peste acel spațiu al vocabularului
- Următorul cuvânt generat este cel care are probabilitatea mai mare

# Arhitectura Transformer



- De asemenea, se folosesc  $N$  straturi de codificare și decodificare
- Fiecare strat primește ieșirea stratului precedent și o rafinează
- Încă o chestie de finețe: am spus că decodicatorul începe să genereze cuvinte pornind de la ultimele cuvinte generate deja, dar la început nu e niciun cuvânt generat
- De fapt, primul cuvânt „generat” de decodicator este token-ul „<START>” sau ceva similar, care să indice că a început generarea
- Primul cuvânt generat efectiv de decodicator (cel după token-ul de start) este obținut din distribuția de probabilități asociată embedding-ului token-ului <START>

**That's it for today!**

## Laboratory 11 – Exercises

Train a Transformer model for machine translation. The dataset is the English-French dataset from the laboratory. Modify the `PositionWiseFFN` class by using 3 hidden layers of 32 units. The dropout probability will be set to 70%. Train the model for 400 epochs and plot the training loss.